

3회차 과제

17기 김지우

01 **배열, 변수, 재귀 함수**

02 **실습과제**

03 **백준 실습 과제**

04 **마무리**

배열

배열은 같은 타입의 변수들을 메모리에 일렬로 나열하여 하나의 이름으로 관리하는 것입니다.

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int arr[5] = {1,2,3,4,5};
6      int sum = 0;
7      int i = 0;
8
9      while (i < 5)
10     {
11         sum +=arr[i];
12         i++;
13     }
14     printf("sum = %d\n", sum);
15
16     return (0);
17 }
```

int arr[5]를 통해 정수 5개를 담을 수 있는 공간을 만듦.

while 반복문을 사용하여 인덱스 i가 0부터 4까지 증가하며 배열의 각 방에 접근

sum += arr[i]는 각 배열 안에 들어있는 숫자들을 하나씩 꺼내 합계 전용 변수인 sum에 넣고 출력.

인덱스 접근

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int arr[5] = {1,2,3,4,5};
6      int sum = 0;
7      int i = 0;
8
9      while (i < 5)
10     {
11         sum +=arr[i];
12         i++;
13     }
14     printf("sum = %d\n", sum);
15
16     return (0);
17 }
18
```

- int arr[5]라고 선언하면 칸은 5개지만, 번호는 0, 1, 2, 3, 4가 부여됨
- 배열의 특정 칸에 접근할 때는 대괄호 [] 안에 인덱스 번호를 넣어 사용
- ex) arr[2] = 10; (3번째 칸에 10을 저장)

n차원 배열 (2차원)

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int arr[2][2] = {{1, 2}, {3, 4}};
6      int i = 0;
7      int j = 0;
8      int sum = 0;
9      while (i < 2)
10     {
11         j = 0;
12         while (j < 2)
13         {
14             sum += arr[i][j];
15             j++;
16         }
17         i++;
18     }
19     printf("합계: %d\n", sum);
20     return 0;
21 }
```

- arr[2][2] 는 2행 2열로 칸을 만듦.
- i 는 행, j 는 열
- while 문 돌면서 j를 0으로 초기화 시켜줘야 함.
- n차원 배열은 뒤에 더 만들면 됨.
 - ex) arr[2][2][2]...

1행 1열	1행 2열
2행 1열	2행 2열

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
jiwoo@gimjiuui-MacBookAir C % gcc main.c
jiwoo@gimjiuui-MacBookAir C % ./a.out
합계 : 10
jiwoo@gimjiuui-MacBookAir C %
```


매개변수와 인자

- 여기서 'x'와 'y', 50, 50 은 인자
- 여기서 'int a'와 'int b'는 매개변수

매개변수는 함수가 실행되기 위해 필요한 데이터의 '형식'을 정의하는 변수

인자는 그 형식에 맞춰 실제로 전달되는 '값'

```
1  #include <stdio.h>
2
3  int add(int a, int b)
4  {
5      return a + b;
6  }
7
8  int main()
9  {
10     int x = 10, y = 20;
11
12     int result = add(x, y);
13     int sum = add(50, 50);
14
15     printf("res = %d\n", result);
16     printf("sum = %d\n", sum);
17
18     return 0;
19 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
• jiwoo@gimjiuui-MacBookAir C % gcc main.c
• jiwoo@gimjiuui-MacBookAir C % ./a.out
res = 30
sum = 100
○ jiwoo@gimjiuui-MacBookAir C %
```

반환값(return)

함수가 실행을 마친 뒤, 자신을 호출한 곳으로 돌려주는 데이터.

```
#include <stdio.h>

int check_even(int num)
{
    if (num % 2 == 0)
        return 1;
    else
        return 0;
}

int main()
{
    int input = 17;
    int result = check_even(input);

    if (result == 1)
        printf("짝수입니다!\n");
    else
        printf("홀수입니다!\n");

    return 0;
}
```

- 반환값 활용
- 17 은 홀수.
- 2를 나누어서 나머지가 0이면 짝수, 0이 아니면 홀수.
- 반환값에 따라서 언제 printf가 실행이 되는지 달라짐.
- return을 통해 값을 돌려줘야만 메인 프로그램이 그 데이터를 받아 다음 단계로 나아갈 수 있음

지역 변수, 전역 변수

```
1  #include <stdio.h>
2
3  int val = 100;
4
5  void test_function(int i, int j)
6  {
7      val += 50;
8      i += 10;
9      j += 10;
10
11     printf("i = %d\n", i);
12     printf("j = %d\n", j);
13     printf("val = %d\n", val);
14 }
```

```
16 int main()  
17 {  
18     int i = 1;  
19     int j = 10;  
20  
21     printf("val = %d\n", val);  
22     test_function(i, j);  
23     printf("i = %d\n", i);  
24     printf("j = %d\n", j);  
25  
26     return 0;  
27 }
```

```
val = 100
i = 11
j = 20
val = 150
i = 1
j = 10
```

- 전역변수는 main 함수에서도 쓸 수 있고, test_function에서도 쓸 수 있다.
- 여기서 'i'와 'j'는 지역변수
- val 은 전역변수

재귀 함수

```
1  #include <stdio.h>
2
3  void countdown(int n)
4  {
5      if (n == 0)
6      {
7          printf("발사!\n");
8          return;
9      }
10     printf("%d...\n", n);
11     countdown(n - 1);
12     printf("발사 다음에 출력됨\n");
13 }
14
15 int main()
16 {
17     countdown(3);
18     return 0;
19 }
```

```
3...
2...
1...
발사!
발사 다음에 출력됨
발사 다음에 출력됨
발사 다음에 출력됨
```

- 재귀 함수에서 가장 중요한 것은 브레이크.
- n이 0이 되면 '발사!'를 출력하고 return을 통해 함수를 종료.
- 만약 재귀함수가 종료조건으로 끝나면, 그 전에 자기 자신을 호출한 시점으로 다시 돌아간다.

표준 라이브러리 함수

- C언어 표준 규격에서 미리 정의하여 컴파일러와 함께 제공하는 함수.
- 즉 이미 잘 만들어진 함수를 사용하기 위해서 그 함수들을 모아둔 라이브러리.
- `stdio`, `unistd`, `stdlib` 등등 있음

사용자 지정 함수

- 특정 기능을 수행하기 위해 개발자가 직접 설계하여 선언하고 정의한 함수
- 그냥 쉽게 말해서 사용자가 직접 설계하고 만든 함수

변수의 스코프

```
1  #include <stdio.h>
2
3  int val = 100;
4
5  void test_function(int i, int j)
6  {
7      val += 50;
8      i += 10;
9      j += 10;
10
11     printf("i = %d\n", i);
12     printf("j = %d\n", j);
13     printf("val = %d\n", val);
14 }
```

- 전역 스코프 : 글로벌 변수처럼 프로그램 종료 전까지 살아있는 변수
- 지역 스코프 : i,j 처럼 test_function이 끝나면 사라지는 변수

```

1  #include <stdio.h>
2
3  int main()
4  {
5      int num;
6      int line;
7      int j, k;
8
9      printf("2단부터 몇 단까지 출력할까요?\n");
10     scanf("%d", &num);
11     printf("한 단에 몇 줄까지 출력할까요?\n");
12     scanf("%d", &line);
13
14     for (k = 1; k <= line; k++)
15     {
16         for (j = 2; j <= num; j++)
17             printf("%d x %d = %d\t", j, k, j * k);
18         printf("\n");
19     }
20
21     return 0;
22 }

```

- 이중 for 문을 사용하여 2단부터 몇 줄을 곱하는 값을 결과를 나타내는 코드와 결과

```

9
한 단에 몇 줄까지 출력할까요?
9
2 x 1 = 2      3 x 1 = 3      4 x 1 = 4      5 x 1 = 5      6 x 1 = 6      7 x 1 = 7      8 x 1 = 8      9 x 1 = 9
2 x 2 = 4      3 x 2 = 6      4 x 2 = 8      5 x 2 = 10     6 x 2 = 12     7 x 2 = 14     8 x 2 = 16     9 x 2 = 18
2 x 3 = 6      3 x 3 = 9      4 x 3 = 12     5 x 3 = 15     6 x 3 = 18     7 x 3 = 21     8 x 3 = 24     9 x 3 = 27
2 x 4 = 8      3 x 4 = 12     4 x 4 = 16     5 x 4 = 20     6 x 4 = 24     7 x 4 = 28     8 x 4 = 32     9 x 4 = 36
2 x 5 = 10     3 x 5 = 15     4 x 5 = 20     5 x 5 = 25     6 x 5 = 30     7 x 5 = 35     8 x 5 = 40     9 x 5 = 45
2 x 6 = 12     3 x 6 = 18     4 x 6 = 24     5 x 6 = 30     6 x 6 = 36     7 x 6 = 42     8 x 6 = 48     9 x 6 = 54
2 x 7 = 14     3 x 7 = 21     4 x 7 = 28     5 x 7 = 35     6 x 7 = 42     7 x 7 = 49     8 x 7 = 56     9 x 7 = 63
2 x 8 = 16     3 x 8 = 24     4 x 8 = 32     5 x 8 = 40     6 x 8 = 48     7 x 8 = 56     8 x 8 = 64     9 x 8 = 72

```



```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4
5  int main()
6  {
7      int i;
8      int random;
9      srand(time(NULL));
10
11     while(1)
12     {
13         random = rand() % 3 + 1;
14         printf("뭘 낼래?\n (1 가위, 2 바위, 3 보)\n");
15         scanf("%d", &i);
16
17         if (i < 1 || i > 3)
18         {
19             printf("1, 2, 3 중에서만 골라줘!\n");
20             continue;
21         }
22         if (i == random)
23         {
24             printf("비겼다 다시!\n");
25             continue;
26         }
27         if (i == 3 && (random == 1 || random == 2) || i == 2 && random == 1)
28         {
29             printf("이겼다! 야호!\n");
30             break;
31         }
32         else
33         {
34             printf("졌다! 으아아앙\n");
35             break;
36         }
37     }
38     return(0);
39 }

```

```

● jiwoo@gimjiuui-MacBookAir C % ./a.out
뭘 낼래?
(1 가위, 2 바위, 3 보)
2
비겼다! 다시!
뭘 낼래?
(1 가위, 2 바위, 3 보)
3
이겼다! 야호!
● jiwoo@gimjiuui-MacBookAir C % ./a.out
뭘 낼래?
(1 가위, 2 바위, 3 보)
1
이겼다! 야호!
● jiwoo@gimjiuui-MacBookAir C % ./a.out
뭘 낼래?
(1 가위, 2 바위, 3 보)
3
이겼다! 야호!
● jiwoo@gimjiuui-MacBookAir C %

```

- while (1) 을 이용하여 무한루프로 scanf를 받는다.
- 이기는 조건 (내가 낸 숫자가 랜덤한 숫자보다 클 때

100%

출력

0%

입력과 계산

100%

조건

0%

반복

0%

빠른 입출력

0%

배열

0%

문자열

0%

함수

조건

5문제 중 5문제 해결

5 1330

두 수 비교하기

구현

폰 사람 수

23만명

평균 시도

2.00회

✓ 해결

5 9498

시험 성적

구현

폰 사람 수

24만명

평균 시도

1.82회

✓ 해결

5 14681

사분면 고르기

구현 · 기하학

폰 사람 수

18만명

평균 시도

1.64회

✓ 해결

5 2753

윤년

구현 · 사칙연산 · 수학

폰 사람 수

21만명

평균 시도

1.93회

✓ 해결

5 2420

사파리월드

구현 · 사칙연산 · 수학

폰 사람 수

3.9만명

평균 시도

2.15회

✓ 해결

반복

7문제 중 1문제 해결

5 2741

N 찍기

구현

폰 사람 수

15만명

평균 시도

1.73회

✓ 해결

3 10872

팩토리얼

수학 · 구현

폰 사람 수

?명

평균 시도

?회

미시도

두 정수 A와 B가 주어졌을 때, A와 B를 비교하는 프로그램을 작성하시오.

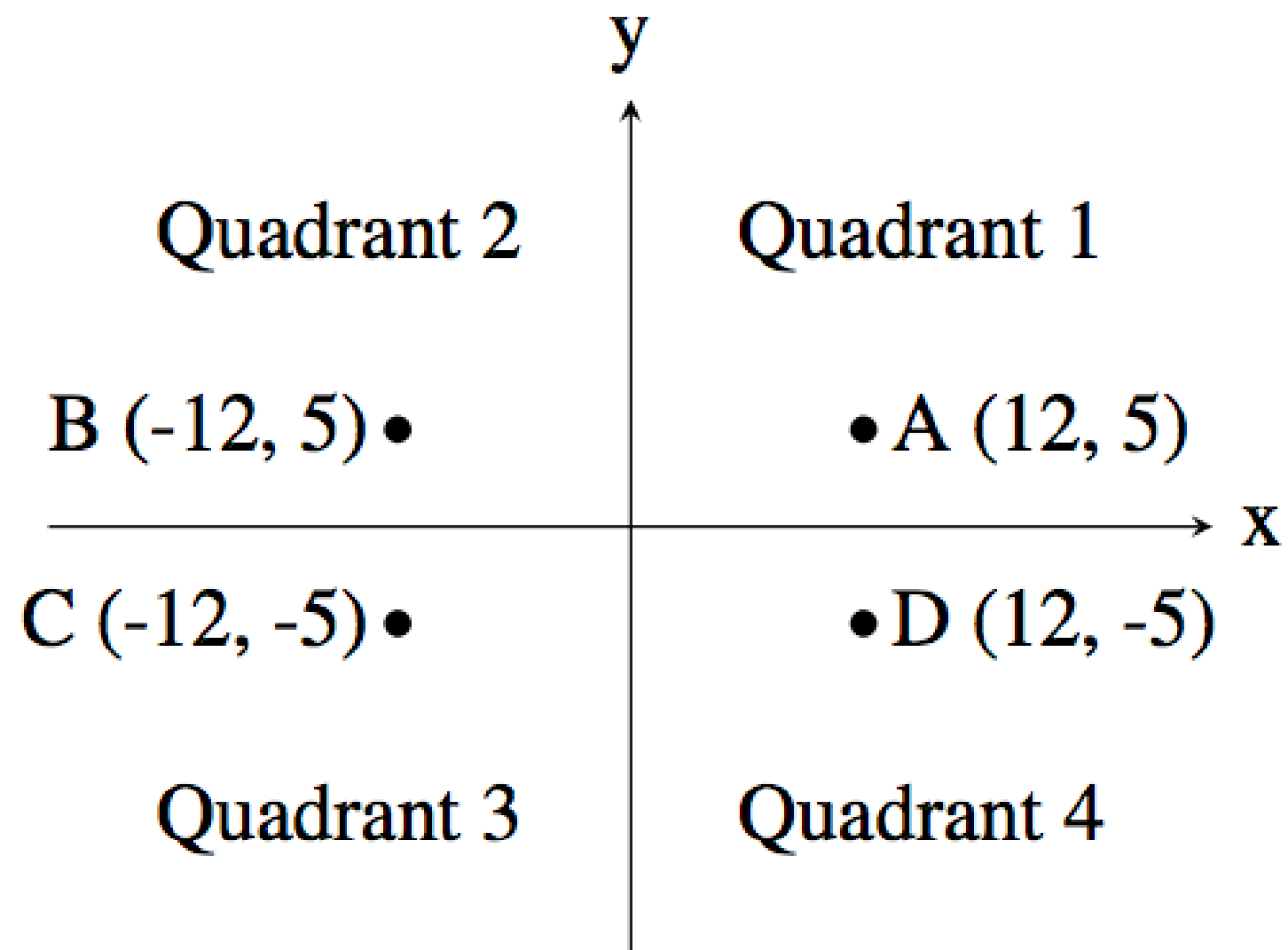
```
1  #include <stdio.h>
2
3  void compare_numbers(int a, int b)
4  {
5      if (a > b)
6          printf(">\n");
7      else if (a < b)
8          printf("<\n");
9      else
10         printf("==\n");
11
12 }
13
14 int main()
15 {
16     int A, B;
17     scanf("%d %d", &A, &B);
18
19     compare_numbers(A, B);
20
21     return 0;
22 }
23
```

a와 b를 비교하여 값을 출력

시험 점수를 입력받아 90 ~ 100점은 A, 80 ~ 89점은 B, 70 ~ 79점은 C, 60 ~ 69점은 D, 나머지 점수는 F를 출력하는 프로그램을 작성하시오.

```
1  #include <stdio.h>
2
3  void print_grade(int score)
4  {
5      if (score >= 90 && score <= 100)
6          printf("A\n");
7      else if (score >= 80)
8          printf("B\n");
9      else if (score >= 70)
10         printf("C\n");
11     else if (score >= 60)
12         printf("D\n");
13     else
14         printf("F\n");
15
16 }
17
18 int main()
19 {
20     int score;
21
22     scanf("%d", &score);
23     print_grade(score);
24
25     return 0;
26 }
```

입력 점수를 비교하여 성적 출력



입력받은 정수를 비교하여 출력

```
1  #include <stdio.h>
2
3  void find_quadrant(int x, int y)
4  {
5      if (x > 0 && y > 0)
6          printf("1\n");
7      else if (x < 0 && y > 0)
8          printf("2\n");
9      else if (x < 0 && y < 0)
10         printf("3\n");
11     else
12         printf("4\n");
13 }
14
15
16 int main()
17 {
18     int x, y;
19
20     scanf("%d", &x);
21     scanf("%d", &y);
22     find_quadrant(x, y);
23
24     return 0;
25 }
```

연도가 주어졌을 때, 윤년이면 1, 아니면 0을 출력하는 프로그램을 작성하시오.

윤년은 연도가 4의 배수이면서, 100의 배수가 아닐 때 또는 400의 배수일 때이다.

**4의 배수이거나, 100의 배수가 아닐때,
또는 400의 배수일 때 윤년**

```
1  #include <stdio.h>
2
3  int check_leap_year(int year)
4  {
5      if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0))
6          return 1;
7      else
8          return 0;
9  }
10
11 int main()
12 {
13     int input_year;
14
15     scanf("%d", &input_year);
16     printf("%d\n", check_leap_year(input_year));
17
18     return 0;
19 }
20
```

첫째 줄에 두 도메인의 유명도 N과 M이 주어진다. ($-2,000,000,000 \leq N, M \leq 2,000,000,000$)

출력

첫째 줄에 두 유명도의 차이 ($|N-M|$)을 출력한다.

예제 입력 1 [복사](#)

-2 5

예제 출력 1 [복사](#)

7

```
1  #include <stdio.h>
2
3  long long get_absolute_diff(long long n, long long m)
4  {
5      long long diff = n - m;
6
7      if (diff < 0)
8          return -diff;
9      return diff;
10 }
11
12 int main()
13 {
14     long long N, M;
15
16     scanf("%lld %lld", &N, &M);
17     printf("%lld\n", get_absolute_diff(N, M));
18
19     return 0;
20 }
```

두 수의 차 의 절대값을 출력하는 문제

int 는 범위보다 작기 때문에 long long을 사용

자연수 N이 주어졌을 때, 1부터 N까지 한 줄에 하나씩 출력하는 프로그램을 작성하시오.

그냥 입력한 값보다 작은 만큼 1부터 출력

```
1  #include <stdio.h>
2
3  void print(int n)
4  {
5      int i = 1;
6
7      while (i <= n)
8      {
9          printf("%d\n", i);
10         i++;
11     }
12 }
13
14 int main()
15 {
16     int N;
17
18     scanf("%d", &N);
19     print(N);
20
21     return 0;
22 }
23
```


THANK YOU